From Sequence to Subcelluar: Deep Learning Architectures for Protein Localization

Gerald Ekechukwu, Harrison Ooi, Chukwuemeka Okoli, Hari Patchigolla
Georgia Institute of Technology
gekechukwu3@gatech.edu, hooi3@gatech.edu, cokoli9@gatech.edu , hpatchigolla3@gatech.edu

Abstract

We address the problem of predicting protein subcellular localization from amino acid sequences using deep learning. Accurate localization prediction is critical for drug discovery, disease diagnosis, and understanding cellular mechanisms, yet experimental methods are time-consuming and costly. While recent deep learning approaches have shown promise, challenges remain in modeling biological sequence patterns directly from primary sequences. We systematically compare Transformer-based models, Convolutional Neural Networks (CNNs), and Bi-directional Long Short-Term Memory (Bi-LSTM) networks for this task. Our dataset, curated from UniProt and focused on nucleus, cytoplasm, and cell membrane localizations, underwent extensive preprocessing to balance class distributions. Experimental results show that the CNN-based model achieved the highest test accuracy (75.18%), while the Bi-LSTM model achieved the lowest test loss, indicating better-calibrated confidence. The Transformer model underperformed relative to CNNs and Bi-LSTMs, likely due to limited dataset size and shorter sequence lengths. Our findings demonstrate that CNNs are particularly effective for extracting biologically meaningful patterns from protein sequences, and suggest future directions combining convolutional and attention-based architectures for improved localization prediction.

1. Introduction/Background/Motivation

We tried to predict where inside a cell a protein will end up, based only on the protein's sequence of building blocks (amino acids). Proteins carry out their jobs in very specific parts of the cell, and knowing where they go helps scientists better understand their function. Our goal was to build models that can make this prediction quickly and accurately without needing expensive and slow laboratory experiments. Scientists use experimental methods like imaging and biochemical techniques to find where proteins are located inside cells. These methods are accurate but very slow, costly, and difficult to apply to thousands of proteins at once. Some computational methods now exist, using deep learning models that look at protein sequences or images, but few studies have systematically compared different deep learning models side-by-side when using only raw sequence data. Many models also rely heavily on additional information like evolutionary profiles, which can be expensive to compute and not always available.

Recent research has explored various deep learning approaches for protein subcellular localization and function prediction. Liao *et al*. [3] combined amino acid sequences with evolutionary features using a BLSTM-CNN model. Du *et al*. [2] developed SecProCT, integrating capsule networks and transformers based solely on sequence data. Sharma *et al*. [5] incorporated sequence, domain, and interaction data using a hybrid CNN-BiGRU-attention model with pretrained embeddings. Su *et al*. [6] leveraged deep image features and adaptive threshold learning for localization tasks. Wang and Wei [7] introduced MSTLoc, combining multi-scale feature extraction, CNNs, and Vision Transformers to handle imbalanced multi-label classification. Together, these studies highlight the growing effectiveness of deep learning in modeling protein localization and function.

If we can accurately predict protein locations from sequence alone, researchers can more quickly discover the roles of new or poorly understood proteins. This would speed up drug discovery, help identify new disease markers, and improve how scientists design targeted therapies. It would also help in understanding diseases like cancer, where proteins are often found in the wrong place inside cells. We used protein sequence data collected from UniProt, a large, publicly available database of protein information. The data included labeled examples of proteins and their known subcellular locations. We carefully cleaned the data by removing duplicates and low-quality entries, balanced the number of examples across the three most common compartments (nucleus, cytoplasm, and cell membrane), and split the dataset into training, validation, and test sets. Each protein sequence varied in length, with

most sequences between a few hundred to a few thousand amino acids.

2. Data Extraction and Preprocessing

Data for this project was extracted from UniProt. More specifically, Uniprot has 20,421 annotated human proteins, however, for this study we only used the amino acid sequence, and the corresponding cell compartments the protein is a part of. While this may not seem like a large enough dataset for a deep learning study, many state-of-the-art models for protein localization prediction use a similar frequent of samples, see more in Table 1 below.

Author	Year	Protein Samples
Liao et al.	2021	3,106
Almagro Armenteros et al.	2017	13,858 proteins initially, reduced to 8,464
Wei Du et al.	2021	Blood secretory: 760 training, 206 testing
Yu-Hang Zhang et al.	2022	4,986

Table 1. Protein samples used for similar studies

Initially, the raw data was highly imbalanced, with certain subcellular compartments like the Nucleus, Cell and Cytoplasm heavily overrepresented compared to others like the Endoplasm, Mitochondrion, and Secreted. In addition, there were duplicate sequences, low-confidence annotations, and inconsistent label formats across databases with missing values as well. We took significant steps to preprocess the data in a format that is ready for machine learning such as performing data cleaning, removing duplicate values, normalization, after which we selected the top three subcellular locations as many of the other locations did not have much representation. Before model training, data processing was needed to ensure that the dataset was usable. The first step was to clean the raw data which included removing row entries that contained missing values (e.g. NaNs) to prevent inconsistencies during model training. After the removal of NaNs, the team conducted a frequency analysis to understand the distribution of subcellular locations within the dataset. A simple frequency diagram showed that the data labeling was imbalanced. Specifically, only the top three locations had a well-balanced amount of data. These locations were:

- Nucleus
- Cytoplasm
- Cell Membrane

The top three locations were used because attempting to model all 10 classes would have introduced severe class imbalance and potentially lead to poorer performance when compared to a better-balanced dataset.

3. Approach

Before developing models for protein subcellular localization, we reviewed relevant literature to understand current modeling techniques, preprocessing strategies, and challenges. Insights from this review guided us to design three approaches each targeting different aspects of sequence patterns:

- Transformer-based model
- Convolutional-based model
- Bi-Directional LSTM model

Throughout the project, we iteratively refined our methods through experimentation and group discussions, leading to a comprehensive evaluation of deep learning models for this prediction task.

3.1. Transformer-Based Model

Figure 1 shows the Transformer-based model architecture developed in this project. Protein sequences, represented as strings of amino acid characters, were used as input. A fixed vocabulary of the 20 standard amino acids was defined, and each amino acid character in a sequence was mapped to a unique numerical index. A special padding token (`<pad>`) was added to handle variable sequence lengths.

To enable batch processing and efficient computation, all sequences were padded or truncated to a maximum length. Sequences shorter than the maximum length were padded with the padding token index.

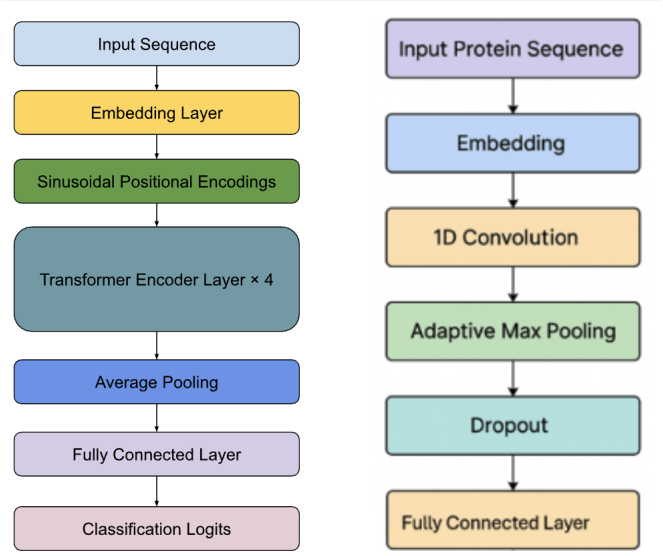


Figure 1. Transformer-Based Model Architecture on the left, CNN-based model on the right

A Transformer-based neural network model was implemented in PyTorch. The architecture consists of the following components:

1. **Embedding Layer:** Maps each amino acid index (including padding) to a dense vector of size 256. These embeddings are learned during training to capture biochemical and functional similarities.
2. **Sinusoidal Positional Encoding:** Injects information about the position of each amino acid within the sequence, complementing the self-attention mechanism.
3. **Transformer Encoder Layers:**
    - Attention heads weigh the importance of different amino acids while a position-wise feed-forward network provides non-linear transformations. Dropout regularization is applied to prevent overfitting.
4. **Average Pooling:** Aggregates amino acid representations into a single fixed-size vector.
5. **Fully Connected Layer:** Maps the pooled representation to output logits for Nucleus, Cytoplasm, and Cell membrane classifications.

3.1.1 Hyperparameter Values

The following hyperparameters, selected through grid search and manual tuning, were used:

- EMBEDDING_DIM = 256
- NHEAD = 8
- NUM_ENCODER_LAYERS = 4
- DIM_FEEDFORWARD = 512
- DROPOUT_RATE = 0.05
- MAX_SEQ_LEN = 2500
- BATCH_SIZE = 128
- NUM_EPOCHS = 20
- LEARNING_RATE = 0.0003

Based on analysis of the dataset, a `MAX_SEQ_LEN` of 2500 was chosen, covering approximately the 98.61st percentile of sequence lengths, thus minimizing information loss from truncation.

3.1.2 Training Setup

The model was trained using the Adam optimizer with weight decay and a learning rate of 0.0003, with a weight decay factor of 1×10^{-4}. Cross-entropy loss was used as the objective function, configured with class weights calculated as the inverse frequency of each location to address class imbalance.

Training was conducted for 20 epochs on a CUDA-enabled GPU. Mini-batches of size 128 were shuffled at the beginning of each epoch. A ReduceLROnPlateau scheduler monitored the validation loss and reduced the learning rate by a factor of 0.1 if no improvement was observed for 8 consecutive epochs, with a minimum learning rate of 1×10^{-6}.

Model performance was monitored using training and validation loss and accuracy. The final evaluation on the held-out test set used the model weights with the best validation loss.

3.2. Convolution-Based Model

The neural network architecture presented in this section is designed to predict the subcellular localization of proteins using only their primary amino acid sequences. This model leverages a convolutional neural network (CNN) structure, which is particularly effective at extracting local patterns from sequential data. The goal is to learn discriminative features from amino acid sequences that can inform accurate classification into predefined subcellular compartments.

Figure 1 shows the CNN-based model architecture. The input to the model is a protein sequence, where each amino acid is first transformed into an integer index using a predefined vocabulary of the 20 standard amino acids. These indices are passed through an embedding layer, which maps each amino acid to a dense vector representation. This allows the network to capture subtle similarities between amino acids, contributing to biological functions and properties.

Next, the output of the embedding layer is processed through a 1-dimensional convolutional layer with 128 filters and a kernel size of 5, enabling the model to detect meaningful local motifs within the sequence. A ReLU activation function is applied to introduce non-linearity.

Following convolution, the model employs an adaptive max pooling layer to produce a fixed-size output regardless of sequence length. Dropout regularization with a rate of 0.5 is applied to prevent overfitting. Finally, a fully connected layer maps the learned features to class logits corresponding to the subcellular locations.

This CNN-based architecture strikes an effective balance between model complexity and interpretability. It is shallow enough for efficient training yet powerful enough to capture critical local sequence features necessary for accurate subcellular localization prediction. The CNN-based design is inspired by successes in text classification and bioinformatics, where local dependencies are critical (e.g., Alley *et al*. [1]; Min *et al*. [4]).

3.2.1 Hyperparameter Values

The model's hyperparameters were tuned using random search, with the following optimal configuration:

- Embedding dimension: 64
- Dropout rate: 0.5
- Weight decay: 1×10^{-5}
- Learning rate: 0.0001
- Batch size: 64
- Number of epochs: 50

An embedding dimension of 64 was selected to balance representational power and computational efficiency. The

dropout rate of 0.5 effectively regularized the model during training. A conservative learning rate of 0.0001 helped ensure stable convergence with the Adam optimizer.

3.2.2 Training Setup

The model was trained using the PyTorch framework, configured to run on a GPU when available. The Adam optimizer was used to update model weights, optimizing the cross-entropy loss function appropriate for multi-class classification. The dataset was initially split into a 90:10 ratio for training and testing. The training set was further divided into 80% training and 20% validation. Each epoch consisted of a forward pass, loss computation, backward pass, and parameter updates. During training, dropout was applied, and both loss and accuracy metrics were monitored for training and validation sets to assess generalization and detect potential overfitting.

3.3. Bi-Directional LSTM Model

The datasets consisted of protein sequences paired with their corresponding subcellular location labels. Given the wide variation in protein sequence lengths (ranging from 2 to 9902 characters), designing a model capable of handling such diversity was a significant challenge. To address this, the problem was framed as a sequential learning task, treating protein sequences similarly to language data, thereby preserving the positional and contextual relationships between amino acids. Due to the sequential nature of the data, Long Short-Term Memory (LSTM) networks were considered a suitable choice.

However, instead of a standard LSTM, we employed a Bi-directional LSTM (BiLSTM) to allow the model to process sequences in both forward and backward directions. This bidirectional context captures information from the beginning, middle, and end of sequences, which is crucial for accurately identifying subcellular localization signals that may appear anywhere within a protein sequence. Figure 2 illustrates the BiLSTM-based network architecture. The model starts with an embedding layer that transforms the input protein sequence into dense vector representations. These embeddings are passed to the BiLSTM layer, and the resulting sequence representations are further processed through an attention mechanism to focus on important regions. Dropout regularization is applied before feeding into a fully connected layer that outputs classification logits corresponding to the target subcellular locations. Cross-entropy loss was used to optimize classification performance.

3.3.1 Hyperparameter Values

To optimize the model's performance, a grid search was conducted over key hyperparameters including embedding dimension, hidden dimension, dropout rate, learning rate, and batch size. These values were chosen to strike a balance between model complexity, convergence stability, and generalization ability given the relatively small and variable-length dataset.After 32 iterations of grid search, the optimal configuration identified is summarized as follows:

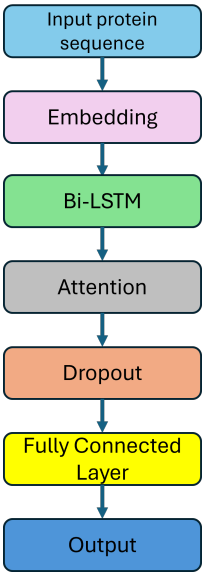


Figure 2. Bi-directional LSTM Model Architecture.

- Embedding dimension: 128
- Hidden dimension: 128
- Dropout rate: 0.6
- Weight decay: 1×10^{-5}
- Learning rate: 0.0005
- Batch size: 16
- Epochs: 10
- Number of layers: 2

The model's hyperparameters were carefully selected to optimize validation accuracy and generalization. An embedding layer was employed to transform the amino acid sequence into a 128-dimensional dense vector space and to allow the model to learn the improved representations beyond just one-hot encodings. This embedding allows the model to capture potential semantic and structural similarities between the variety of amino acids which is important to classify the sequences into biological sequences. The hidden dimension was set to 128. This was done to balance the model's complexity and prevent overfitting risk. This will provide sufficient capacity to help the model learn meaningful dependencies within the protein's sequence. To further enhance the model's ability to correlate the protein's sequence to its corresponding localization the Bi-LSTM was configured to have two stacked layers. This was done to enable deeper feature extractions across the protein sequences.

A dropout rate of 0.6 was used. This was done to regular-

ize the network and reduce the chances of overfitting with consideration of the size of the dataset. The weight decay was set to 1e-5 and was applied with the optimization to penalize large weights. The learning rate was set to 0.0005. This was done to ensure that there is a stable convergence during gradient descent. The model was also trained to 10 epochs after hyperparameter tuning.

3.3.2 Training Setup

Training was conducted using the Adam optimizer with a learning rate of 0.0005 and an L2 regularization (weight decay) term of 1×10^{-5} to prevent large weight magnitudes. The model was trained using a cross-entropy loss function, which is standard for multi-class classification tasks, to measure the difference between predicted class probabilities and true labels.

The dataset was initially split into a 90:10 ratio for training and testing, with an 80:20 split applied again to create a validation set from the training data. The training loop included forward passes, loss computation, backpropagation, and weight updates across all epochs, with dropout enabled during training to regularize the network.

4. Experiments and Results

4.1. Transformer Results

Figure 3 shows the plot of training and validation loss and accuracy for the Transformer model. The results demonstrate the effectiveness of applying a Transformer network with learned amino acid embeddings for predicting protein subcellular localization from sequence. The model was able to learn meaningful patterns, achieving a validation accuracy of over 70% and similar performance on the unseen test set for the task of classifying proteins into Nucleus, Cytoplasm, and Cell membrane. The learned embedding layer likely contributed to capturing the biochemical similarities and relationships between different amino acids that are relevant for localization signals.

The Transformer's self-attention mechanism proved capable of modeling the complex, potentially non-local dependencies within protein sequences that constitute these signals. The training strategy incorporating class weighting was important for handling the expected imbalance among the three target locations. The learning rate scheduler, although it did not trigger a steep reduction within the 20 epochs due to consistent (albeit small) improvements in validation loss, provided a mechanism for adaptive learning rate adjustment if a plateau were to occur. The early stopping strategy based on validation loss ensured that the best performing model iteration on unseen data was captured and used for final evaluation.

On the unseen test dataset, the Transformer model achieved 71.22% accuracy. One reason the performance could potentially be higher is that the model architecture, while effective, may not fully capture complex hierarchical or local structural patterns within protein sequences that could aid in more accurate classification.

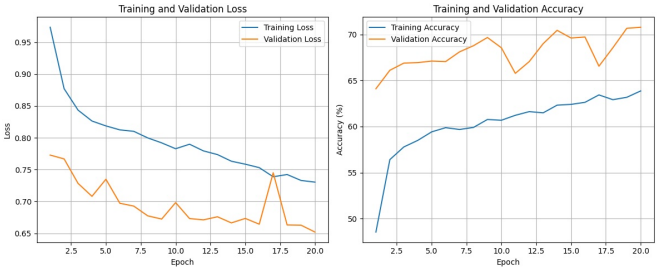


Figure 3. Training and Validation Curves for Transformer Loss and Accuracy.

4.2. Convolutional-Based Model Results

The model was trained over 30 epochs using cross-entropy loss and optimized with the Adam optimizer. The model demonstrated stable convergence throughout the training process, as reflected in both the loss and accuracy curves for the training and validation datasets. As shown in the loss curve in Figure 4, the training loss exhibited a consistent decline from an initial value above 1.28 to approximately 0.76 by epoch 30. Similarly, the validation loss decreased from around 1.05 to approximately 0.78. The near-parallel behavior of the training and validation loss curves indicates good generalization without significant overfitting. No divergence between training and validation losses was observed, and the curves plateaued together after approximately epoch 25.

The training accuracy improved from an initial value near 36% to around 65%, while the validation accuracy rose from approximately 48% to 60% over the course of training. A slight gap emerged between training and validation accuracies after epoch 20, but it remained small (about 5%), indicating controlled levels of overfitting. The model achieved a test accuracy of 75.18% and a test loss of 0.6401 on the unseen dataset. The model's generalization to unseen data suggests that it learned meaningful patterns rather than merely memorizing examples. The use of dropout regularization during training likely contributed to this. The convergence behavior and validation performance indicate that 30 epochs was an appropriate training duration.

4.3. Bi-LSTM Results

4.3.1 Training vs Validation Accuracy and Loss

As shown in the accuracy curve in Figure 5, both the training and validation accuracies exhibited an upward trend during the early epochs. This suggests successful learn-

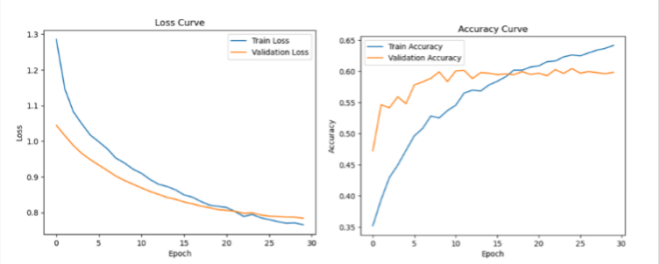


Figure 4. Training and Validation Loss & Accuracy Curve for Convolutional-Based Model.

ing of valuable patterns based on the training data. However, as training progressed, the validation accuracy began to plateau while the training accuracy continued to rise. This divergence indicates that the model started to overfit, memorizing training patterns rather than improving performance on unseen data.

As shown in the loss curve in Figure 5, the training loss consistently decreased in a near-linear fashion, suggesting that the optimizer effectively minimized the objective function. In contrast, the validation loss decreased initially but plateaued and hovered between the 70%-80% range, indicating that the model had reached its capacity with the given hyperparameters.

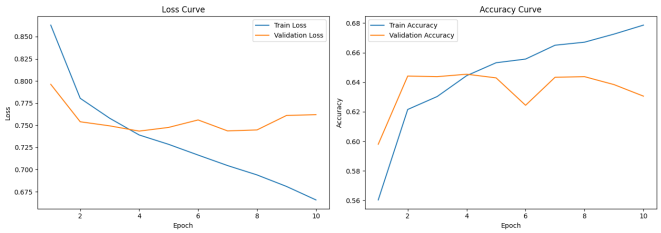


Figure 5. Training and Validation Loss & Accuracy Curve for Bi-LSTM Model.

4.3.2 Overall Model Performance

After training on 90% of the dataset, the Bi-LSTM model achieved a 73.86% classification accuracy on the remaining 10% test set. While the model was successful in generalizing beyond the training data, the divergence of the training and validation accuracy curves suggests mild overfitting.

Despite this, achieving a 73.86% test accuracy demonstrates the Bi-LSTM architecture's effectiveness given the relatively small dataset of three subcellular locations. Future enhancements could involve stronger regularization or data augmentation to improve performance further.

4.4. Model Comparison

In this study, three different deep learning models were compared for their performance on the protein sequence classification task. Table 2 shows the results on the hold-out (test) dataset.

Model	Test Loss	Test Accuracy
Transformer-Based Model	0.5973	71.22%
Convolutional-Based Model	0.6401	75.18%
Bi-LSTM Model	0.5733	73.86%

Table 2. Test loss and accuracy across all three models.

The Convolutional-based model achieved the highest test accuracy (75.18%), outperforming the Transformer and Bi-LSTM models. This suggests that convolutional architectures were particularly effective at capturing local sequence patterns relevant to subcellular localization. In terms of test loss, the Bi-LSTM model achieved the lowest value (0.5733), followed by the Transformer-based model (0.5973), and the CNN model had the highest loss (0.6401). Lower loss values imply better-calibrated confidence, while accuracy measures raw prediction correctness. Therefore, while the Bi-LSTM was better calibrated, the CNN achieved higher classification correctness.

The Transformer model, though effective, may not have reached its full potential due to the relatively small dataset and shorter sequence lengths, limiting its ability to leverage long-range dependencies. Overall, when prioritizing accuracy, the Convolutional-based model emerges as the best performer. If confidence calibration were prioritized, the Bi-LSTM model would be a strong candidate.

5. Conclusion

In this study, we explored the performance of Transformer-based, CNN-based, and Bi-LSTM-based deep learning models for the task of predicting protein subcellular localization from sequence data. Each model exhibited distinct strengths, with the CNN-based model achieving the highest accuracy. The superior performance of the CNN model can be attributed to its ability to capture local sequence motifs, which appear to be critical for subcellular localization prediction. Meanwhile, the similar overall performance of all three models suggests that despite architectural differences, each was able to extract biologically meaningful patterns.

Future work could involve hybrid models that combine CNN feature extraction with Transformer attention mechanisms to capture both local and long-range dependencies. Expanding the dataset, incorporating evolutionary information, and using finer-grained labels could also further improve model performance. This study demonstrates that deep learning architectures can effectively learn to predict protein localization from primary sequence data, with convolutional networks providing a strong baseline for future work. The code can be referenced in the github location.

6. Work Division

Summary of contributions are provided in Table 3.

References

[1] Ethan C. Alley, Grigory Khimulya, Surojit Biswas, Mohammed Alquraishi, and George M. Church. Unified rational protein engineering with sequence-only deep representation learning. *Nature Methods*, 16(12):1315–1322, 2019.

[2] Wei Du, Xuan Zhao, Yu Sun, Lei Zheng, Ying Li, and Yu Zhang. Secproct: In silico prediction of human secretory proteins based on capsule network and transformer. *International Journal of Molecular Sciences*, 22(16), 2021.

[3] Zhijun Liao, Gaofeng Pan, Chao Sun, and Jijun Tang. Predicting subcellular location of protein with evolution information and sequence-based deep learning. *BMC Bioinformatics*, 22(10):515, 2021.

[4] Seonwoo Min, Byunghan Lee, and Sungroh Yoon. Deep learning in bioinformatics. *Briefings in Bioinformatics*, 18(5):851–869, 07 2017.

[5] Lavkush Sharma, Akshay Deepak, Ashish Ranjan, and Gopalakrishnan Krishnasamy. A novel hybrid cnn and bigru-attention based deep learning model for protein function prediction. *Statistical Applications in Genetics and Molecular Biology*, 22(1), 2023.

[6] Ran Su, Linlin He, Tianling Liu, Xiaofeng Liu, and Leyi Wei. Protein subcellular localization based on deep image features and criterion learning strategy. *Briefings in Bioinformatics*, 22(4):bbaa313, 07 2021.

[7] Fengsheng Wang and Leyi Wei. Multi-scale deep learning for the imbalanced multi-label protein subcellular localization prediction based on immunohistochemistry images. *Bioinformatics*, 38(9):2602–2611, 02 2022.

Student Name	Contributed Aspects	Details
Harrison Ooi	Data Extraction, Preprocessing, Model development, Experimentation	Preprocess the dataset and trained the Bi-LSTM model. Performed several experiments.
Gerald Ekechukwu	Data Preprocessing, Model development, Experimentation	Implemented CNN-based model and performed several experiments.
Chukwuemeka Okoli	Model development, Experimentation	Trained RNN and LSTM and performed several experiments.
Hari Patchigolla	Data Extraction, Preprocessing, Model development, Experimentation	Extracted the dataset, Preprocess the dataset and trained the Transformer model. Performed several experiments.

Table 3. Contributions of team members.